DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 79

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c, where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$1 \leq N, Q \leq 10^5$
$1 \leq u, v \leq N$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers $N \; and \; Q$ respectively
- Second line: A string s of length N (where the ith character of s represents the character stored in node i)
- Next $N-1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

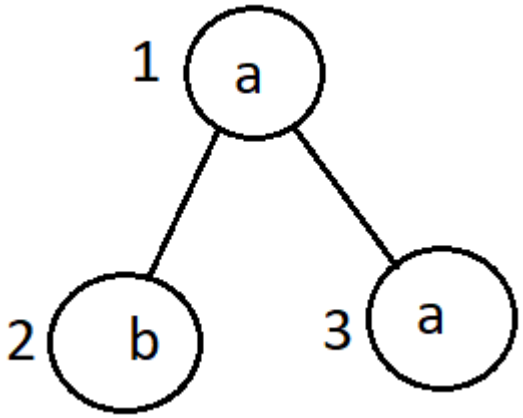
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that.



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 1
aba
1 2
1 3
1 a
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
4 1
wsx
1 2
1 3
1 1
1 w
```

EXPECTED OUTPUT

```
10
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(Q--)
```

Test Case 2

KEYWORD

```
for(i = 0;i<N-1;i ++)
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
4	330	49

Code

C subtree_alphabet_count.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head[MAXN];
Edge pool[2 * MAXN];
int pool_sz = 0;
char labels[MAXN];

int counts[MAXN][26];

void add_edge(int u, int v) {
    pool[pool_sz].to = v;
    pool[pool_sz].next = head[u];
    head[u] = &pool[pool_sz++];

    pool[pool_sz].to = u;
    pool[pool_sz].next = head[v];
    head[v] = &pool[pool_sz++];
}

void dfs(int u, int p) {
    counts[u][labels[u] - 'a'] = 1;
    for (Edge* e = head[u]; e; e = e->next) {
        if (e->to != p) {
            dfs(e->to, u);
            for (int c = 0; c < 26; c++) {
                counts[u][c] += counts[e->to][c];
            }
        }
    }
}

int main() {
    int N, Q;
    if (scanf("%d %d", &N, &Q) != 2) return 0;

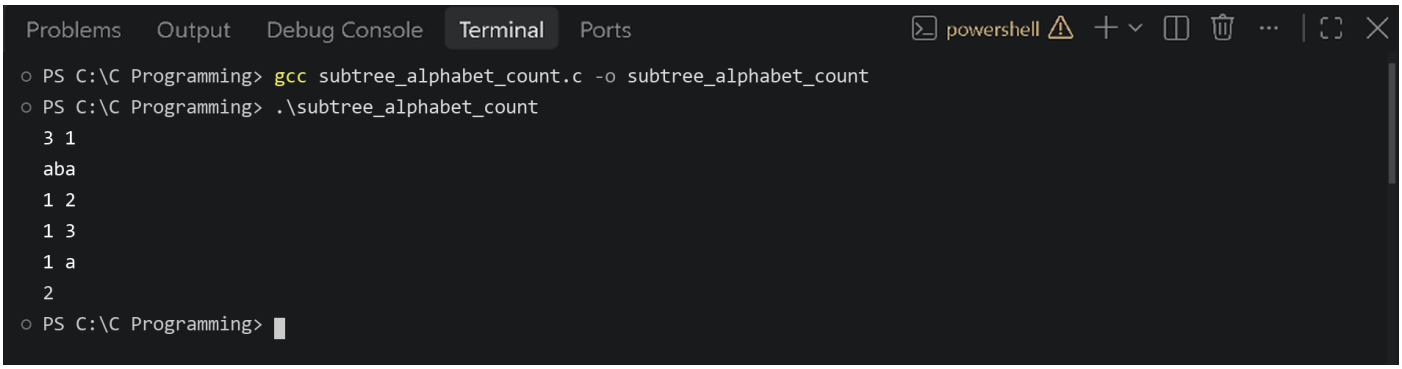
    char s[MAXN];
    scanf("%s", s);
    for (int i = 1; i <= N; i++) {
        labels[i] = s[i - 1];
        head[i] = NULL;
    }

    int i;
    for(i = 0;i<N-1;i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v);
    }

    dfs(1, 0);

    while(Q--) {
        int u;
        char c;
        scanf("%d %c", &u, &c);
        printf("%d\n", counts[u][c - 'a']);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc subtree_alphabet_count.c -o subtree_alphabet_count
PS C:\C Programming> .\subtree_alphabet_count
3 1
aba
1 2
1 3
1 a
2
PS C:\C Programming>